

Understanding Query Cost

To optimize a query, we first need to understand how to calculate its cost.



A "cost" represents the actual system resources—specifically **CPU time, memory, and Disk I/Os**—required to execute a request.



The database optimizer's primary goal is to find the access plan with the **absolute lowest total cost**.

Pages

- Depending on the storage model (**row store vs. column store**), rows are stored and read in logical **pages**.
- Each page has a fixed size (e.g., **8KB in PostgreSQL, 16KB in MySQL**).
- Assume each page holds 3 rows: $100 \text{ rows} \div 3 \approx$ **34 pages**.

row_id	emp_id	emp_name	emp_dob	emp_salary
1	101	Omar	6/14/1991	\$85,000
2	202	Youssef	11/8/1985	\$145,000
3	303	Karim	2/21/1994	\$190,000
...	...	...	...	...
100	101000	Daniel	9/5/2001	\$275,000

Page 0

1, 101, Omar,
6/14/1991, \$85,000
2, 202, Youssef,
11/8/1985, \$145,000
3, 303, Karim,
2/21/1994, \$190,000

Page 1

(Rows 4, 5, 6) ...

Page 2

(Rows 7, 8, 9) ...

...

Page 34

More rows ...
1000, 101000, Daniel,
9/5/2001, \$275,000

I/O

- A **disk I/O** (input/output) operation is a request to move data between disk and memory. For database systems, the number of disk I/Os required by a query is an important measure of its cost.
- A database reads data in pages rather than individual rows. A page read brings all of the rows stored in that page into memory.
- Because an I/O brings an entire page into memory, the other rows on that page are available without requiring additional page reads.
- Reducing the number of pages a query needs to access generally reduces its I/O cost, which is why database systems try to minimize unnecessary page reads.
- Not every page read requires a physical disk access. If the required page is already in the operating system's cache, the database can retrieve it from memory instead.
- Cached reads are still not free. The database may still need to process, interpret, and materialize the page's contents before the query can use the rows.

Page 0

1, 101, Omar,
6/14/1991, \$85,000
2, 202, Youssef,
11/8/1985, \$145,000
3, 303, Karim,
2/21/1994, \$190,000

Page 1

(Rows 4, 5, 6) ...

Page 2

(Rows 7, 8, 9) ...

...

Page 34

More rows ...
1000, 101000, Daniel,
9/5/2001, \$275,000

The Heap

- The **heap** is the data structure that stores the table's pages one after another.
- It holds the actual data: every row and every column.
- Traversing the heap is **expensive**: finding a few rows means reading a lot of data.
- That is why we need **indexes**: they tell us exactly which page(s) of the heap to pull.

Heap

Page 0

1, 101, Omar,
6/14/1991, \$85,000
2, 202, Youssef,
11/8/1985, \$145,000
3, 303, Karim,
2/21/1994, \$190,000

Page 1

(Rows 4, 5, 6) ...

Page 2

(Rows 7, 8, 9) ...

...

Page 34

More rows ...
1000, 101000, Daniel,
9/5/2001, \$275,000

Example: The Data File

- Data file: **EMPLOYEE**(NAME, SSN, ADDRESS, JOB, SAL, ...)
- Suppose that:
 - Record size **R = 150 bytes**
 - Block size **B = 512 bytes**
 - Number of records **r = 30,000**
- Then we get:
 - Blocking factor $Bfr = B \div R = 512 \div 150 = 3 \text{ records/block}$
 - File blocks $b = r / Bfr = 30,000 / 3 = 10,000 \text{ blocks}$
- Theoretical cost of a full scan to find one record:
 - Worst case: **10,000 disk I/Os** (read every block)
 - Average case: **5,000 disk I/Os** ($b / 2 = 10,000 / 2$)

Data file

Block 0

Record 1
Record 2
Record 3
450 of 512 bytes used

Block 1

Records 4–6

Block 2

Records 7–9

...

Block 9,999

Records 29,998–30,000

Example: The SSN Index

SSN index

- For an index on the **SSN** field, assume:
 - SSN field size $V_{SSN} = \mathbf{9 \text{ bytes}}$
 - Record pointer size $P_R = \mathbf{7 \text{ bytes}}$
- Then we get:
 - Index entry size $R_i = V_{SSN} + P_R = 9 + 7 = \mathbf{16 \text{ bytes}}$
 - Blocking factor $Bfr_i = B \text{ div } R_i = 512 \text{ div } 16 = \mathbf{32 \text{ entries/block}}$
 - Index blocks $b_i = r / Bfr_i = 30,000 / 32 \approx \mathbf{938 \text{ blocks}}$
 - Binary search on the index: $\log_2 938 \approx \mathbf{10 \text{ block accesses}}$

Index block 0

32 entries × 16 bytes
SSN (9 B) + pointer (7 B)
→ points into data blocks

Index block 1

Entries 33–64

Index block 2

Entries 65–96

...

Index block 937

Last entries, up to record
30,000

Comparing Access Cost

- Linear search on the unordered file averages $b/2 = 10,000 / 2 = \mathbf{5,000}$ **block accesses**.
- If the file were ordered by SSN, binary search would need $\log_2 10,000 \approx \mathbf{14}$ **block accesses**.
- Binary search on the SSN index needs $\log_2 938 \approx \mathbf{10}$ **block accesses**.
- The index cuts the cost **500×** compared with a linear scan.

5,000 block accesses
Linear search · unordered file

14 block accesses
Binary search · ordered file

10 block accesses
Binary search · SSN index

Before B+ Trees: Single-Level Indexes

Everyone starts indexing with **B+ trees**. But a B+ tree is the answer to a problem, and the problem starts with **the simplest index: one sorted list over a sorted file**.

Keeping data **physically sorted** makes every insert **ripple through the file**. ISAM avoided the ripple with **overflow chains**; B+ trees fixed it with **local splits**. PostgreSQL sidesteps it entirely by keeping the table as an **unordered heap**.



But first, a quick **challenge**

You know B+ trees. But how do databases use them?

Every engine builds on the same B+ tree, but each one decides **where the rows live** and **what an index entry points to**.

Dense vs sparse

Is your primary key index dense or sparse?

Index types

What's the real difference between a clustering and a secondary index?

Clustering vs clustered

Are they the same thing?

Real engines

Does PostgreSQL store your table sorted?

Dense or sparse: one entry per record, or per block

Dense index

- One entry for **every record** in the data file
- Works whether or not the file is sorted
- Our SSN index: 30,000 records → **30,000 entries, 938 blocks**

Sparse index

- One entry for **every block**, pointing at its first (anchor) record
- Only works if the file is **sorted on the index field**
- Same file sorted by SSN: 10,000 blocks → **10,000 entries, 313 blocks**

The textbook types follow from this

- **Primary**: sparse, on a file sorted by a key field
- **Clustering**: sparse, on a file sorted by a non-key field, one entry per distinct value
- **Secondary**: dense, on any other field; the file can be in any order

Problems with single level indexes

#	Where	Exact operation	What happens
1	Data file	Insert "Adams, Mike" into block 2, which is already full	Block 2's last row moves to block 3, block 3's last row moves to block 4, and so on to the end of the file. Each block is read and rewritten.
2	Data file	Delete "Adams, Robin" and physically close the gap	The reverse of #1: rows from the following blocks shift back to fill the hole , block by block.
3	Index file	Update anchors after #1 or #2	Blocks 3, 4, 5... now start with different names, so each of their index entries needs its key rewritten . This is a direct result of the data-file shift.
4	Index file	Insert a new index entry when the data file gains a new block	The index is sorted, so the new entry must go in its correct position, and index entries after it shift across index blocks (the same ripple as #1, inside the index).
5	Index file	Delete an index entry when a data block becomes empty	Index entries after it shift back to close the gap (the reverse of #4).

The solution: a multilevel index with a **fixed structure** and **overflow areas** for new rows, so a change stays in one block instead of rippling. Next: how ISAM does this.

Clustering $\neq$ clustered: two vocabularies

Term	Where it comes from	What it means
Clustering index	Elmasri & Navathe's taxonomy	A specific <i>type</i> : a sparse index on a file ordered by a non-key field, one entry per distinct value
Clustered index	Vendor docs, and other textbooks too	A <i>storage decision</i> : the table's rows are stored in that index's order

From here on, **clustered** means the storage decision: the table's rows sit on disk in the index's order. A table can have only one.

[CS186 Berkeley notes: B+ trees, clustered and unclustered](#)

[Microsoft Learn: clustered and nonclustered indexes](#)

[Wikipedia: Database index](#)

[PostgreSQL docs: CLUSTER](#)

Made by **AbdulRahman Tamer**

Only InnoDB and SQL Server sort the rows

	Heap	Clustered table (rows stored in B+ tree order)
Clustered	<i>Not possible.</i> PostgreSQL only gets close: CLUSTER is a one-time operation, and later updates aren't stored in index order.	InnoDB (always) and SQL Server (clustered table)
Non-clustered	PostgreSQL (every index) and SQL Server heap	InnoDB and SQL Server clustered table : secondary indexes that store the key as the locator

The locator is the difference: a heap index stores a **row pointer** (TID in PostgreSQL, RID in SQL Server); a clustered-table index stores the **key**, so each lookup walks the clustered index a second time.

[PostgreSQL docs: CLUSTER](#)

[MySQL 9.4 manual: clustered index selection](#)

[MySQL 9.7 manual: clustered and secondary indexes](#)

[Microsoft Learn: clustered and nonclustered indexes](#)

Made by **AbdulRahman Tamer**